Device Binding for BBS from ECDSA-PoPs

Anja Lehmann

May 2025

This document gives a brief overview of how pairing-based BBS credentials can be made device-bound to a Secure Element (SE) that can only produce standard ECDSA signatures (over curve P256). BBS signatures are currently the most studied and widely used multi-message signature scheme. These signatures allow for an efficient zero-knowledge proof (ZKP) of a signature over hidden attribute values, which immediately leads to a highly efficient anonymous credential system. BBS signatures have already been ISO standardized in ISO/IEC 20008-2:2013, which also includes a lightweight method for device-binding, assuming the Secure Element can compute a Schnorr-signature on a pairing-friendly curve. This method divides the ZKP for the credential presentation into two parts: the SE provides only a proof-of-possession (PoP) of the device key, whereas the user's host device computes the rest of the ZKP of the BBS credential, see e.g., [CDL16, DL24]. If such a direct support is not possible, one can instead bridge BBS credentials to standard ECDSA signatures for the proof-of-possession, which comes at (slightly) larger costs.

Disclaimer. This write-up is meant as a high-level summary, explaining that pairing-based anonymous credential using BBS or PS signatures can be made device-bound from existing secure elements that only support ECDSA signatures over P256. Further work to detail the full protocol and perform a formal security analysis is needed.

High-Level Idea. The user's attribute-based credentials will be issued as BBS credential $cred = \mathsf{BBS.Sign}(isk, (a_1, \ldots, a_n, dpk))$ for an issuer key isk, user attributes $a_1, \ldots, a_n$, and a device public key dpk. The latter is an ECDSA public key of the user's Secure Element (SE). In the concrete instantiation this will take two attributes to include the ECDSA public key in the BBS credential. The corresponding ECDSA secret key dsk is only available through an $\mathsf{ECDSA.Sign}$ interface of the SE. Enrollment of the device key during issuance can remain unchanged, i.e., the current mode of using a simple proof-of-possession for issuance can still be used.

To present the credential, the user creates a BBS ZKP π_{BBS} (over a pairing-friendly curve, such as BLS12-381) for the attributes she wants to reveal. The BBS ZKP must not reveal the device public key dpk, but instead reveals only a committed public key $com_{dpk} = \mathsf{Commit}(dpk)$ and proves that com_{dpk} contains the dpk from the certified credential $cred$. In addition, the SE is invoked to create a proof-of-possession through an ECDSA signature $\sigma := \mathsf{ECDSA.Sign}(dsk, nonce)$ for a fresh session nonce under dsk. Instead of revealing the signature, the user now produces an additional ZKP π_{PoP}, proving that she has a signature σ that is valid under the committed key com_{dpk}:

$$\pi_{\mathsf{PoP}} = \mathsf{ZKP}\{(\sigma, dpk) : \mathsf{ECDSA.Vf}(dpk, nonce, \sigma) = 1 \ \wedge \ com_{dpk} = \mathsf{Commit}(dpk)\}.$$

The user sends π_{BBS} and π_{PoP} to the relying party, which verifies that both are correct, and valid for the same com_{dpk}.

Note on π_{PoP}. As σ is a one-time signature computed for a fresh nonce, the signed message as well as parts of the ECDSA signature (the r value) can be revealed in the zero-knowledge proof, without harming the desired unlinkability. This significantly reduces the size and complexity of the proof compared to ZKPs over multi-use attribute ECDSA credentials, where the signed message and full ECDSA signature must remain hidden.

Existing Solutions. The idea has been proposed in the context of ring- and group signatures as zkAttest [FLM21] by Faz-Hernández et al. and subsequently been improved by Celi et al. [CLR23]. Combining this with BBS credentials has been proposed and implemented by Amrein/Ubique in the SPRIND EUDI Funke [Amr25, Har25]. Their implementation currently takes around 800ms to generate proofs, which have a size of 150KB. Further, recent works on using zkSNARKs to prove knowledge of ECDSA legacy credentials include highly efficient instantiations for such proof-of-possessions too – again leveraging the observation that the signed message and part of the signature can be revealed for that proof [PPZ24, WOS+25]. For instance, the protocol by [WOS+25] takes 1.6ms for the proof generation and produces proofs of 400B. Likewise, also the ECDSA-ZKP protocol of Google [FS24] contains such a proof-of-possession subpart that can be used in combination with BBS. This will likely result in a solution that is around 1-2 magnitudes faster and more compact than the ZKP based entirely on ECDSA signatures.

Note that the highly efficient protocol by Woo et al. [WOS+25] gains its efficiency by moving to a weaker privacy setting, as anonymity therein is based on the DL-assumption. Thus, privacy can retroactively be broken once quantum computers exist. In stronger ZKP variants, privacy typically holds unconditionally. It is likely that the protocol by Woo et al. [WOS+25] can be turned into a quantum-annoying version, without sacrificing efficiency. That means that privacy still is based on the DL assumption, but every de-anonymization of a ZKP requires breaking a fresh DL instance. Moving to a perfectly or statistically private version is possible by relying on the CDLS [CLR23] protocol or using a lightweight zkSNARK for parts of the protocol.

Apart from zkAttest [FLM21] and CDLS [CLR23], the other protocols also need a bridging for the correct representation in a BBS credential. Note that the ECDSA public key is only available in the P256 curve, and we need to include the public key in the BBS credential which encodes all attributes in the scalar-field of $\mathbb{G}_1$ of the pairing-friendly curve. This can be done by using the x, y coordinates of the ECDSA public key and embedding them as $g_1^{dpk.x} g_2^{dpk.y}$ in the BBS credential. For proofs that work for a committed public key in P256, this either has to get adapted to work directly with a commitment $com_{\mathsf{BBS}} = g_1^{pk.x} g_2^{pk.y} h^o$ or one has to additionally prove the following statement

$$\pi_{\mathsf{PoP}'} = \mathsf{ZKP}\{(dpk, o, o') : com_{dpk} = dpk \cdot h^o \ \wedge \ com_{\mathsf{BBS}} = g_1^{dpk.x} g_2^{dpk.y} h^{o'}\}.$$

Comparison to full-fledged zkSNARK. Compared to computing a zkSNARK over an ECDSA attribute-credential (and ECDSA-PoP), e.g., [FS24, PPZ24, WOS+25] the BBS-ECDSA approach has several advantages: zkSNARKs that are efficient enough for real-world deployment currently require highly-optimized circuits and proof systems. Generic approaches are at least 1-2 magnitudes slower. This makes zkSNARKs in the short/midterm mostly applicable for scenarios where hardcoding the circuit and proof statement does not limit the flexibility and extendability of the system. This is the case here, as the statement to be proven via the zkSNARK is independent of the attribute statements proven on top, which is done via a simple and flexible Schnorr proof. Further, as the PoP ECDSA signature is a one-time signature computed for a fresh nonce, the signed message as well as parts of the signature (the r value) can be revealed in the zero-knowledge proof, which significantly reduces the size and complexity of the proof. In particular, no statement over a hash function must be proven, which is often the most complex part within the circuit. In contrast to a full-fledged zkSNARK built entirely upon ECDSA credentials, the BBS-ECDSA approach requires changes to the issuer's credential signing though, but significantly reduces the overall complexity of the proof system.

Device-Binding for Pairing-Free Solution. BBS can also be used in a pairing-free yet publicly-verifiable variant, as recently proposed as BBS# [DDKT25] and Server-Aided Anonymous Credentials (SAAC) [CAHLT25]. BBS# also proposes a solution for device-binding from ECDSA-signatures, relying on a re-randomization feature of ECDSA signatures and keys. If the security of this approach can be formally vetted, this yields an immediate ECDSA-binding solution for any credential system based on *pairing-free* DL-based multi-message signatures. This is significantly simpler and more compact than the solution suggested here – but only works in the pairing-free setting. The pairing-free variants come with inherent trade-offs in terms of deployment

though, as they either require batch issuance or online verification, both requiring access to the issuer's secret key for every presentation.

References

[Amr25] Patrick Amrein. https://github.com/UbiqueInnovation/zkattest-rs, 2025.

[CDL16] Jan Camenisch, Manu Drijvers, and Anja Lehmann. Anonymous Attestation Using the Strong Diffie Hellman Assumption Revisited *TRUST 2016* https://eprint.iacr.org/2016/663, 2016.

[DL24] Cordian Daniluk and Anja Lehmann. Device Binding for BBS Signatures *SPRIND Funke Note, available upon request.* 2024.

[CAHLT25] Rutchathon Chairattana-Apirom, Franklin Harding, Anna Lysyanskaya, and Stefano Tessaro. Server-aided anonymous credentials. Cryptology ePrint Archive, Paper 2025/513, https://eprint.iacr.org/2025/513, 2025.

[CLR23] Sofía Celi, Shai Levin, and Joe Rowell. CDLS: proving knowledge of committed discrete logarithms with soundness. *Progress in Cryptology – AFRICACRYPT 2024.* https://eprint.iacr.org/2023/1595, 2023.

[DDKT25] Nicolas Desmoulins, Antoine Dumanois, Seyni Kane, and Jacques Traoré. Making BBS anonymous credentials eIDAS 2.0 compliant. https://eprint.iacr.org/2025/619, 2025.

[FLM21] Armando Faz-Hernández, Watson Ladd, and Deepak Maram. Zkattest: Ring and group signatures for existing ECDSA keys. *Selected Areas in Cryptography – SAC 2021.* https://eprint.iacr.org/2021/1183, 2021.

[FS24] Matteo Frigo and Abhi Shelat. Anonymous credentials from ECDSA. Cryptology ePrint Archive, Paper 2024/2010, https://eprint.iacr.org/2024/2010, 2024.

[Har25] Lovesh Harchandani. hhttps://github.com/docknetwork/crypto/tree/main/equality_across_groups, 2025.

[PPZ24] Christian Paquin, Guru-Vamsi Policharla, and Greg Zaverucha. Crescent: Stronger privacy for existing credentials. Cryptology ePrint Archive, Paper 2024/2013, https://eprint.iacr.org/2024/2013, 2024.

[WOS+25] Anna P. Y. Woo, Alex Ozdemir, Chad Sharp, Thomas Pornin, and Paul Grubbs. Efficient proofs of possession for legacy signatures. *IEEE Security & Privacy 2025.* https://eprint.iacr.org/2025/538, 2025.